Advanced Tokenization Methods for Molecular Foundation Models

Student: Nguyen Cao Duy (A0258078R)
Project ID: H437020
Project Supervisor: Dr. Bian Yatao
Main Evaluator: Dr. Chen Yu

Outline

1. Introduction
    a. Overview
    b. Objectives
2. Literature Review
    a. Simplified Molecular Input Line Entry System (SMILES)
    b. Tokenization Taxonomy
3. Current Progress
    a. Methodology
    b. Experiment Results
4. Research Plan

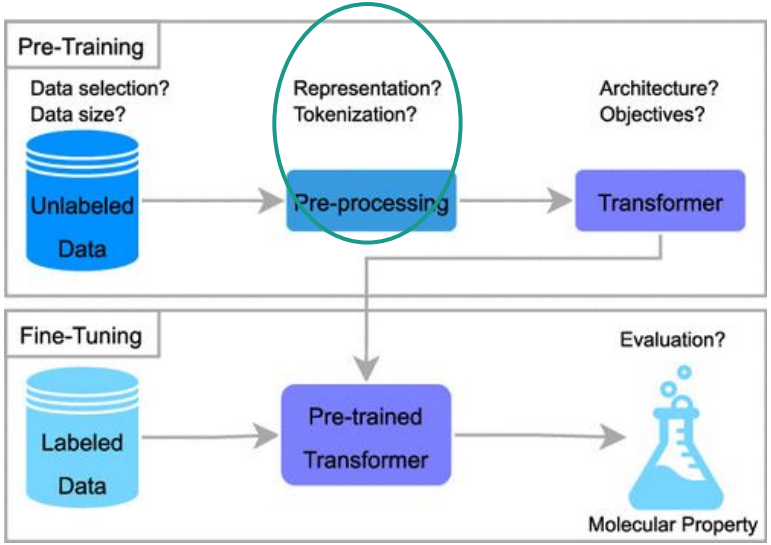
Introduction

Overview

- Transformers from NLP have been adopted in computational chemistry.
  - Molecule Property Predictions
  - Molecule Generation
- Use SMILES strings (1D representation of molecules) to learn the rules of chemistry.
- **The Problem:** How we split these strings (tokenization) can affect how the model learns.

Objectives

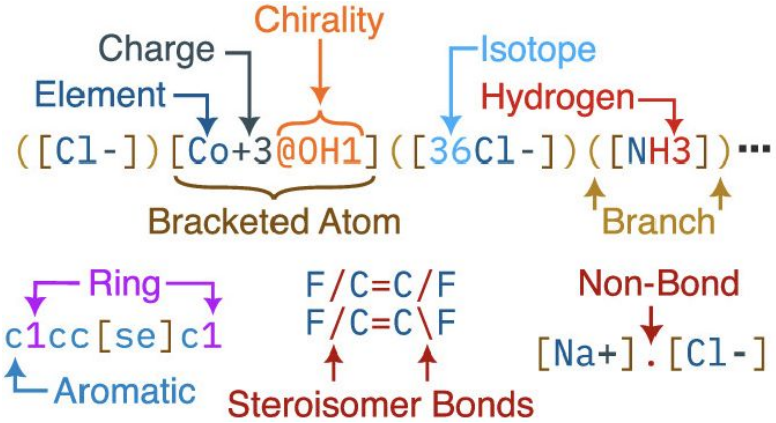
1. **Benchmark Existing Tokenizers:**
   - Intrinsic metrics: fertility, UNK frequency.
   - Extrinsic metrics: downstream performance.
2. **Design Improved Tokenization:**
   - Experiment different modifications to existing tokenization schemes.

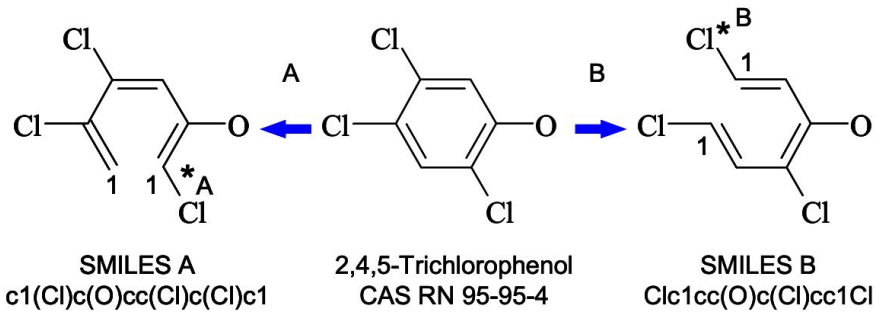


Literature Review

Simplified Molecular Input Line Entry System (SMILES)

- DFS traversal on 2D molecular graph.
- A molecule can have multiple valid SMILES representation.





Tokenization Taxonomy

- Character
- Glyph
- Atom
- Fragment

Character-based Tokenizers

- Most simple, < 128 ASCII letters
  - **Issue 1:** Each molecule maps to a long sequence of tokens (high fertility), waste computation
  - **Issue 2:** Some SMILES symbol is > 1 character (Sc could represent the element Scandium, or it could represent a Sulfur atom S next to an aromatic carbon c)
- Fix **issue 1** use BPE or PCATT to build up frequently occurring larger tokens (increasing the vocabulary size to get lower fertility)
  - **Byte Pair Encoding:** starting from an initial vocabulary, iteratively combining the most frequently co-occurring adjacent token pairs into larger tokens
  - **Partition Cover Approach to Tokenization:** starting from an initial vocabulary, iteratively adding tokens that maximize the marginal contribution to the corpus coverage **(more general solution than BPE)**

Glyph-based Tokenizers (SMIRK)

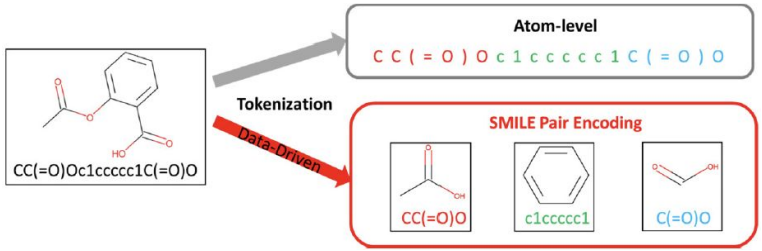
- Fix **issue 2** of character-based tokenizers.
- Initial vocab size is 165.
- Can also run BPE or PCATT to fix **issue 1**.
- **Issue 3:** purely statistical, merged tokens might not be chemically meaningful structures

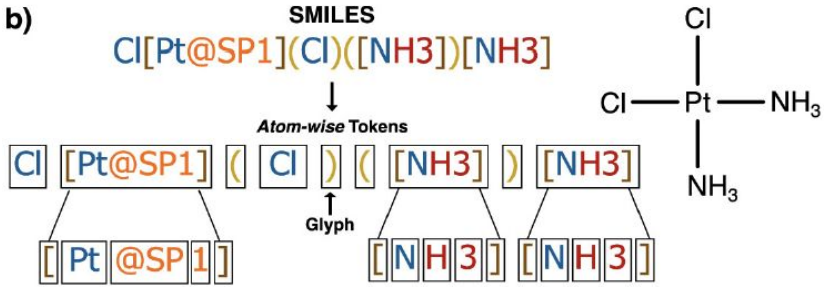
# 1	$ 2	% 3	(4	) 5	* 6	+ 7	- 8	. 9	/ 10	0 11	1 12	2 13	3 14	4 15	5 16
6 17	7 18	8 19	9 20	: 21	= 22	@ 23	@@ 24	@AL 25	@OH 26	@SP 27	@TB 28	@TH 29	Ac 30	Ag 31	Al 32
Am 33	Ar 34	As 35	At 36	Au 37	B 38	Ba 39	Be 40	Bh 41	Bi 42	Bk 43	Br 44	C 45	Ca 46	Cd 47	Ce 48
Cf 49	Cl 50	Cm 51	Cn 52	Co 53	Cr 54	Cs 55	Cu 56	Db 57	Ds 58	Dy 59	Er 60	Es 61	Eu 62	F 63	Fe 64
Fl 65	Fm 66	Fr 67	Ga 68	Gd 69	Ge 70	H 71	He 72	Hf 73	Hg 74	Ho 75	Hs 76	I 77	In 78	Ir 79	K 80
Kr 81	La 82	Li 83	Lr 84	Lu 85	Lv 86	Mc 87	Md 88	Mg 89	Mn 90	Mo 91	Mt 92	N 93	Na 94	Nb 95	Nd 96
Ne 97	Nh 98	Ni 99	No 100	Np 101	O 102	Og 103	Os 104	P 105	Pa 106	Pb 107	Pd 108	Pm 109	Po 110	Pr 111	Pt 112
Pu 113	Ra 114	Rb 115	Re 116	Rf 117	Rg 118	Rh 119	Rn 120	Ru 121	S 122	Sb 123	Sc 124	Se 125	Sg 126	Si 127	Sm 128
Sn 129	Sr 130	Ta 131	Tb 132	Tc 133	Te 134	Th 135	Ti 136	Tl 137	Tm 138	Ts 139	U 140	V 141	W 142	Xe 143	Y 144
Yb 145	Zn 146	Zr 147	[148	\ 149	] 150	as 151	b 152	c 153	n 154	o 155	p 156	s 157	se 158		

Table 1: Complete SMIRK tokenizer vocabulary of 165 unique tokens (after including the 7 special tokens ([BOS], [EOS], [UNK], [SEP], [PAD], [CLS], [MASK])

Atom-based tokenizers (SPE & APE)

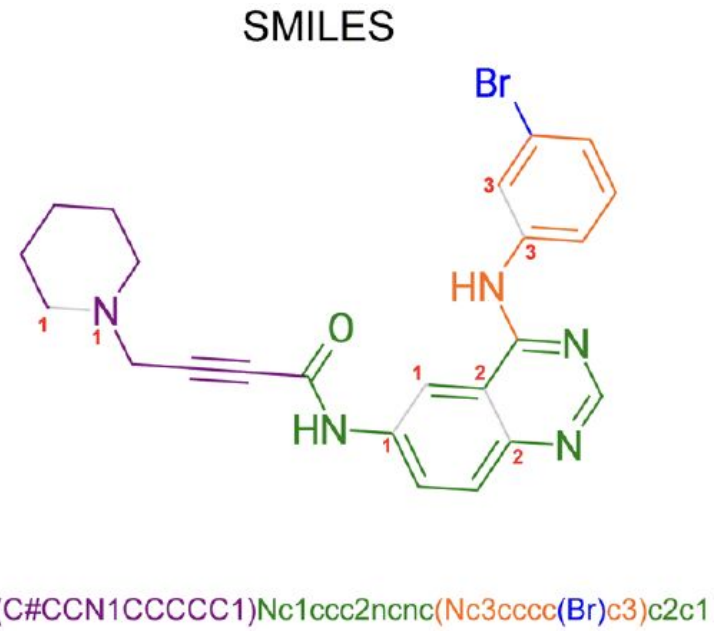
- Treat all atoms as irreducible tokens, then run BPE algorithm.
- **Issue 4:** the initial vocab size can be large (trillions of valid “bracketed atoms”)





SMILES Limitations

- **The problem:** substructures (fragments) in the 2D molecular graph might be separated by long sequences of intervening characters in the 1D SMILES string.
- **The idea:** identify fragments in the molecule and define a 1D string that respect the fragments.

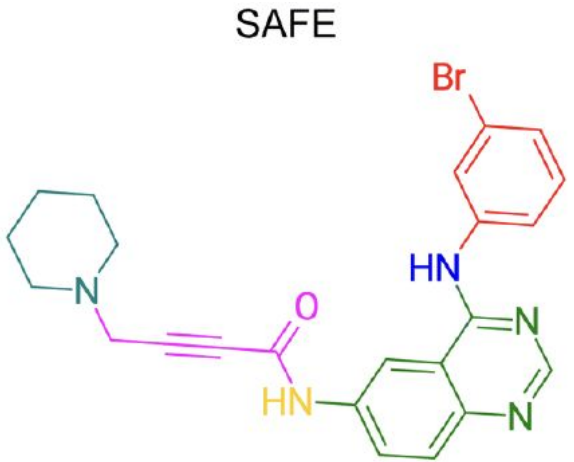


Fragment-based tokenizers

- Different algorithms to slice a molecule into fragments:
  - BRICS
  - MMPA
  - We can also define custom fragmentation rules
- Different fragment-based 1D representations:
  - SAFE
  - tSMILES
  - fragSMILES

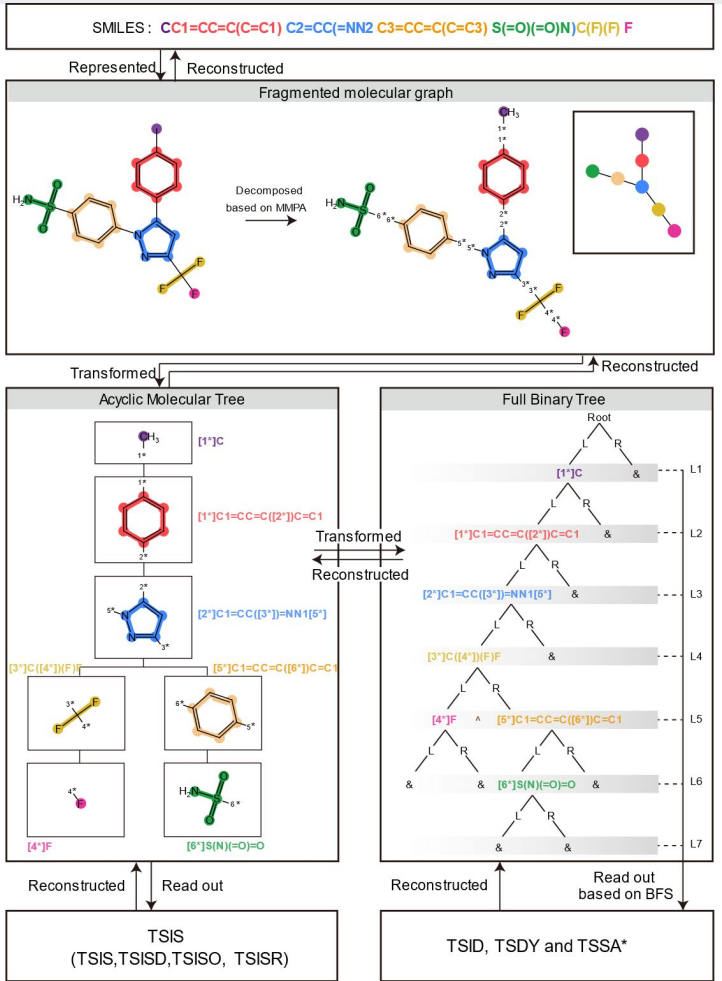
Fragment-based tokenizers (SAFE)

- **Valid SMILES syntax**
- Sequence of fragments separated by non-bonding notations .
- Ring-closure digits to indicate attachment points between disconnected fragments



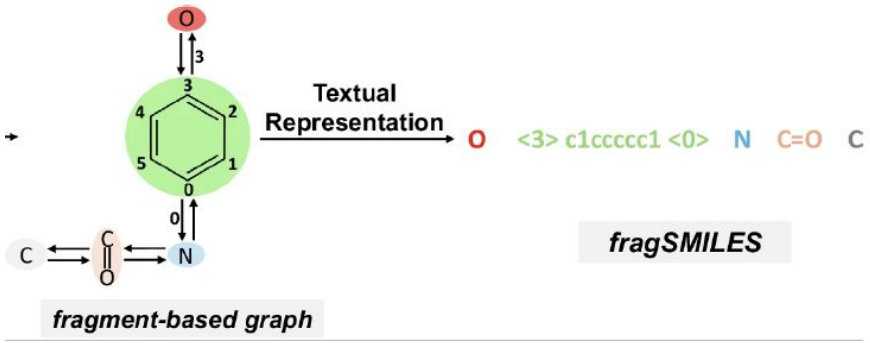
Fragment-based tokenizers (t-SMILES)

- Modified SMILES syntax
- BFS on the full binary tree:
  - TSSA: shared atoms
  - TSID: dummy atoms with ID `[id*]`
  - TSDY: dummy atoms without ID `[*]`
- Different ordering on the spanning tree (using `[id*]` like TSID):
  - TSIS (BFS)
  - TSIS (DFS)
  - TSISO (decreasing fragment size)
  - TSISR (random)
- &: non-fragment node in FBT
- ^: separate adjacent fragments



Fragment-based tokenizers (fragSMILES)

- Modified SMILES syntax
- DFS on the acyclic fragment graph
- `<id>`: Indicating the atom where the bond attaches to the fragment (omitted if only 1 possible attachment point).
- `.` and `()` : Separate adjacent fragments, and wrap around branch closures.



Fragment-based tokenizers

- Treat all fragments as irreducible tokens.
- **The problem:**
  - SAFE and t-SMILES embed the connection syntax directly inside the fragment.
  - The same fragment may be represented differently in the string -> inefficient vocabulary
- fragSMILES avoids this problem by using `<id>` outside the fragments.
- Example:
  - SMILES: `Cc1ccc(cc1)c2ccccc2`
  - SAFE: `c13ccc4cc1.c14ccccc1.C3`
  - fragSMILES: `C.<0>c1ccccc1<3>.<0>c1ccccc1`

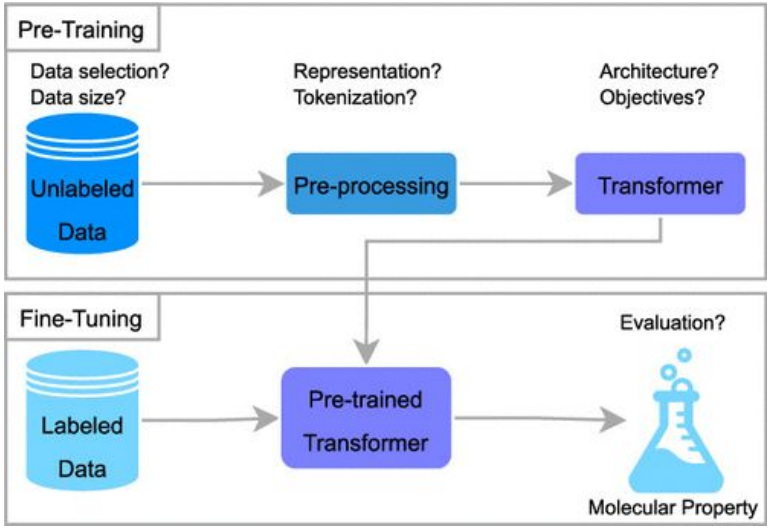
Data-driven fragmentation (PS-VAE)

- **The problem:**
  - General fragmentation algorithm using predefined rules might not work well for all datasets.
  - Can encounter unknown fragments after slicing a molecule in the test set.
- **The idea:**
  - BPE on the 2D molecular graph.
  - Starting from an initial vocabulary of atom nodes, iteratively combining the most frequently co-occurring adjacent subgraphs into larger subgraphs.

Current Progress

Methodology

1. **Tokenizer Training:** 2M PubChem molecules, 5K Vocab.
2. **MLM Pretraining:** RoBERTa (6-layer, 12-head, max length 512), 1M PubChem molecules, 20 Epochs.
3. **Downstream Finetuning:** BBBP and HIV binary classification datasets (MoleculeNet splits), 5 Epochs, ROC-AUC (average 3 trials)
4. **Constant hyperparameters** across all runs to isolate the effect of the tokenizers.



Experiment Results

- Lower-level representations actually performed better on downstream tasks.

Category	Top Configuration	HIV ROC	BBBP ROC
Character	SMILY-SBPE	0.7726	**0.7452**
Glyph	SMILY-SMIRK-SPCATT	**0.7730**	0.7212
Atom	SMILY-SPE	0.7470	0.7292
Fragment (t-SMILES)	TSMILES-TSSA-M	0.7192	0.6867
	TSMILES-TSDY-M	0.7014	0.7002
Fragment (fragSMILES)	FRAGSMILES	0.7506	0.7204

Table 2: Summary of top-performing configurations across the five tokenization categories on downstream HIV and BBBP datasets.

Experiment Results

- Some pre-tokenizing pattern can improve character-based tokenization performance.
- Avoid merging across digits and brackets.
- Ex: `[Cu+2].[O-]S(=O)(=O)[O-]` pre-tokenized to `[Cu+2]`, `.`, `[O-]`, `S`, `(=O)`, `(=O)`, `[O-]`

Configuration	HIV ROC	BBBP ROC
SMILY-BPE	0.7370	0.7154
SMILY-SBPE	0.7726	0.7452
SMILY-SMIRK-GPE	0.7301	0.6850
SMILY-SMIRK-SGPE	0.7360	0.6785

Table 3: Performance comparison of structural pre-splitting configurations on character and glyph-based BPE tokenization.

Experiment Results

- Running character/glyph/atom tokenization on SAFE degraded performance, especially on BBBP.

Configuration	Δ HIV ROC	Δ BBBP ROC
BPE	0.0007	−0.0566
SBPE	−0.0365	−0.0725
SPCATT	0.0036	−0.0797
SMIRK	−0.0049	−0.0573
SMIRK-GPE	−0.0211	−0.0519
SMIRK-SGPE	0.0001	−0.0600
SMIRK-SPCATT	−0.0248	−0.0050
APE	0.0148	−0.0399
SPE	−0.0148	−0.0568

Table 4: Performance delta when transitioning from standard SMILES to SAFE. Negative Δ values indicate that the SAFE strings underperform compared to the original SMILES strings.

Experiment Results

- The choice of fragmentation rule and assembly affects downstream performance

Configuration	Fragmentation Rule	HIV ROC	BBBP ROC
TSSA	BRICS	0.6025	0.6197
TSDY	BRICS	0.6052	0.6270
TSID	BRICS	0.5564	0.5682
TSIS	BRICS	0.5390	0.5660
TSSA-M	MMPA	0.7192	0.6867
TSDY-M use `[*]`	MMPA	0.7014	0.7002
TSID-M use `[id*]`	MMPA	0.6854	0.6746
TSIS-M	MMPA	0.7011	0.6549

Table 5: Performance comparison of t-SMILES configurations evaluating the impact of connection grammar (TSSA/TSDY vs. TSID/TSIS) and molecular fragmentation rules (BRICS vs. MMPA).

Experiment Results

- Train PS-VAE on the same tokenizer-training corpus.
- fragSMILES (using data-driven fragmentation rule) currently *underperform*.
- **Note:** data-driven fragmentation rule may lead to cyclic fragment graph; modified by isolate and collapse all ring structures in the molecule into single nodes prior to fragmentation to ensure acyclic fragment graph.

Configuration	Tokenizer Corpus Size	HIV ROC	BBBP ROC
FRAGSMILES	2,000,000	0.7506	0.7204
PS-FRAGSMILES	20,000	0.7269	0.7117

Table 6: Performance comparison between FRAGSMILES and PS-FRAGSMILES. The PS-FRAGSMILES tokenizers were constrained to a much smaller training corpus due to computational bottlenecks of the PS-VAE library.

Research Plan

Future Work

- Extend PS-VAE:
  - Train on bigger corpus
  - Try on SAFE and t-SMILES
- Modify SAFE and t-SMILES:
  - Design a connection syntax outside of the fragments (e.g., combining `<id>` and `[ID*]`)
- Extend the benchmark:
  - Intrinsic metrics: fertility, UNK rate
  - Other downstream tasks/datasets